



PDF Download
3539491.3539591.pdf
30 March 2026
Total Citations: 3
Total Downloads: 257

 Latest updates: <https://dl.acm.org/doi/10.1145/3539491.3539591>

RESEARCH-ARTICLE

Towards edge-assisted real-time 3D segmentation of large scale LIDAR point clouds

FRASER MCLEAN, The University of Edinburgh, Edinburgh, Scotland, U.K.

LEYANG XUE, The University of Edinburgh, Edinburgh, Scotland, U.K.

CHRIS XIAOXUAN LU, The University of Edinburgh, Edinburgh, Scotland, U.K.

MAHESH MARINA, The University of Edinburgh, Edinburgh, Scotland, U.K.

Open Access Support provided by:

The University of Edinburgh

Published: 27 June 2022

Citation in BibTeX format

MobiSys '22: The 20th Annual
International Conference on Mobile
Systems, Applications and Services
July 1, 2022
Oregon, Portland

Conference Sponsors:
SIGMOBILE

Towards Edge-assisted Real-time 3D Segmentation of Large Scale LIDAR Point Clouds

Fraser McLean, Leyang Xue, Chris Xiaoxuan Lu, Mahesh Marina

School of Informatics, The University of Edinburgh

United Kingdom

s1720557@sms.ed.ac.uk, leyang.xue@ed.ac.uk, xiaoxuan.lu@ed.ac.uk, mahesh@ed.ac.uk

ABSTRACT

Light detection and ranging (LIDAR) has become a cost-effective and accessible sensor for a broad range of embedded devices including mobile phones and drones. Vision applications of these embedded devices require fast and accurate inferences to drive them, while at the same time power consumption should be kept low. Achieving both these requirements is hard due to the size of high quality LIDAR point cloud data streams – significantly larger than vision inputs such as images. The complexity of point cloud segmentation adds further difficulty for achieving high quality, realtime LIDAR data driven vision applications on battery powered embedded devices. We consider edge offloading as a potential approach to reconcile these conflicting requirements. Specifically, we present an experimental characterization study exploring the benefit of edge-assisted LIDAR point cloud segmentation, considering diverse set of embedded devices and state-of-the-art semantic segmentation models. Our results indicate that edge offloading is always beneficial from a device energy efficiency perspective and can also significantly reduce inference latency, especially with compressive edge offloading. These latency improvements however fall short of meeting real-time requirements. We outline a number of potential follow-on research directions to enable edge assisted accurate and real-time LIDAR point cloud segmentation.

CCS CONCEPTS

• **Computer systems organization** → Embedded software; • **Computing methodologies** → *Graphics systems and interfaces*; • **Hardware** → *Power estimation and optimization*.

KEYWORDS

3D point cloud, LAIDAR, machine learning, offloading

ACM Reference Format:

Fraser McLean, Leyang Xue, Chris Xiaoxuan Lu, Mahesh Marina. 2022. Towards Edge-assisted Real-time 3D Segmentation of Large Scale LIDAR Point Clouds. In *6th International Workshop on Embedded and Mobile Deep Learning (EMDL22)*, July 1, 2022, Portland, OR, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3539491.3539591>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EMDL22, July 1, 2022, Portland, OR, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9404-8/22/07...\$15.00

<https://doi.org/10.1145/3539491.3539591>

1 INTRODUCTION

Light detection and ranging (LIDAR) due to its speed of data generation and the accuracy has a significant advantage over photogrammetry based approaches for computer vision applications (e.g., semantic and instance segmentation for real-time autonomous navigation, interactive augmented reality). With advances in solid-state technology, LIDAR is being increasingly integrated into resource constrained, and embedded devices. For example, Apple's recent iPad Pro and iPhone 12 Pro use their on-board LIDAR as the default method for image reconstruction. Lightweight drones are another example that largely rely on LIDAR for land surveying.

Although LIDAR has become a cost-effective and accessible sensor, the heavy computational demands associated with processing high quality LIDAR point cloud data is a hindrance to real-time sophisticated perception tasks such as 3D Segmentation on resource constrained and battery operated devices. Autonomous drones, for example, may require real-time decisions for navigation [22]. This hindrance is becoming increasingly prominent with the emergence and adoption of high-resolution LIDARs. The cutting-edge LIDAR (e.g., RoboSense¹) is able to generate 200,000 points per frame with the point number expected to keep growing. State-of-the-art (SOTA) deep learning based 3D-Segmentation methods (e.g., [9]) can provide high segmentation accuracy, but are known to be computationally expensive and power hungry, rendering them ill-suited for processing large scale point clouds in real-time on resource-constrained platforms. Reducing computational complexity through simplification of raw point cloud data comes at the expense of accuracy. The key unresolved challenge in this context is balancing between high quality, real-time LIDAR data processing and low power consumption on resource constrained devices.

In this paper, for the first time, we examine the potential of edge computing to help address the aforementioned challenge. We explore the benefit of offloading computationally intensive inference operations on LIDAR point cloud data to a nearby edge compute infrastructure, which promises to preserve inference accuracy while relieving the device from the inference related computation and power consumption burden. To this end, we present an experimental characterization study considering semantic segmentation as a representative application of LIDAR point cloud data. Specifically, we experiment with several SOTA deep learning based models and using the widely used LIDAR point cloud Semantic-KITTI [4] dataset. We consider a wide spectrum of embedded devices and an emulated edge infrastructure to study the effect of different inference strategies – on-device processing, edge offloading with and without compression – on inference latency and energy per inference under diverse network conditions.

¹<https://www.roboSense.ai/en>

Our results show that both forms of edge offloading yield significant reductions in energy per inference performance. Edge offloading also provides gains in inference latency relative to on-device processing for severely resource constrained devices whereas improvements are more consistent with compressive edge offloading. While we consider only the common Octree point cloud data compression, these results highlight the importance of point cloud data communication cost to inference latency. However, ensuring real-time inference is still a challenge which remains to be addressed. Real-time can be considered to be anything over 30Hz [13], providing a challenging inference latency target of at most 33ms. To that end, we present several directions for building on the work presented in this paper towards edge assisted real-time LIDAR point cloud segmentation.

The focus on 3D point cloud data sets our work apart from prior research on edge offloading (e.g., [5, 11, 25]) that mainly dealt with image data. In our case, not only are inference tasks computationally intensive but the associated point cloud data is significantly larger than image data, the latter making the communication cost a bigger concern for edge offloading.

2 BACKGROUND

2.1 3D Segmentation

Segmentation is a process in which each point/pixel in a point cloud/image is assigned to an object category to enable high-level understanding of a particular scene, and is key to many LIDAR based vision tasks. Segmentation tasks can be divided into three major categories: Semantic, Instance and Part (or Hierarchical) Segmentation. Modern approaches to segmentation of 3D point clouds fall into two main categories: 1) Projection based methods (e.g., [2]); and 2) Point based methods (e.g., [9]). Projection methods give a point cloud more structure by projecting them into several 2D images. Each of these images are analyzed separately, and a final semantic interpretation is determined by fusing the images back together [6]. This may result in information loss as some of the geometric structure of the point clouds may not be retained. Point based methods, on the other hand, use point clouds as direct inputs to their permutation invariant models. This allows all geometric information to be retained and reduces the amount of pre-processing required on point clouds. However, as the data is unstructured, finding neighboring points (e.g., by k-nearest neighbors (KNN)) is very expensive.

2.2 Edge Computing

Edge-assisted DNN. Edge computing aims to take the processing burden off the device, while keeping the overall latency to a minimum by leveraging nearby edge infrastructure. A typical approach for deep neural network (DNN) related edge offloading involves partitioning the DNN model so that computationally expensive layers can be executed at the edge [5, 11]. Additionally, intermediate results of DNNs can be compressed for offloading to reduce communication costs [25]. Prior works on DNN related edge offloading have focused on the mature image and video processing field, and not on the larger data sized, and rapidly evolving, 3D point clouds field as in our case. Moreover, the unstructured nature of point cloud files prevents them from being directly used within

Device	NVIDIA		
	Jetson Nano	Jetson TX2	AGX Xavier
CPU Capability	4× 1.48GHz	2× 2.26GHz 4× 1.48GHz	8× 2.26GHz
GPU (TFLOPs)	0.472	1.33	5.5
Memory	4GB	8GB	16GB
Power	1.8W	2.2W	5.3W

Table 1: Hardware specifications of embedded devices. Memory is shared between CPU and GPU.

the standard convolutional neural networks (CNNs), which are not input permutation invariant.

Edge-assisted Point Cloud Processing. DNN related offloading often requires retraining and modification of neural network architectures for early exit. Conversely, efficient processing of point cloud data can be achieved by model partitioning and compressive sensor data offloading which is adaptive to various workloads and network conditions. Perception latency and coverage can then be improved by offloading LIDAR data to edge servers for collaborative inference [26]. Although computational graph based model partitioning, together with dynamic scheduling, is general enough to be applied on any type of neural network, CNN workloads are often the intended use case [3, 28].

2.3 Efficient Segmentation Models

Tiny DNN models can help to lower model runtime memory requirements such that they are well suited for edge devices and embedded GPUs. Whilst realtime service level objectives are more likely attainable, this comes with the inevitable cost of compromising accuracy in the region of 5% up to even 20% in segmentation tasks [14]. Tiny models can be both efficient and accurate in single object detection tasks, however, they often fail to generalize to semantic segmentation [20]. Several attempts have been made to improve inference efficiency without degrading accuracy, however, the realtime inference is far from being satisfied [24]. The challenge of efficient on-device segmentation not only depends on DNN architecture, but also input data size [8], and when it comes to urban scale segmentation, the frame size becomes a major bottleneck.

3 METHODOLOGY

Here we describe our experimental characterization methodology, covering the point cloud segmentation models considered, dataset used, inference strategies under study, evaluation metrics and the experimental testbed setup.

3.1 Testbed Setup

We choose a range of NVIDIA embedded GPU from low-end to high-end (Table 1) to emulate different types of devices capability, where the computational power is limited and power consumption has to be low to prolong battery life. Baseline power readings were determined based on each of the boards respective internal power monitors, thirty minutes after startup, with no processing load. The devices of choice allow for power efficient processing in a small form-factor without massively sacrificing performance.

Our testbed (Figure 1) emulates end-to-end communication between devices (embedded GPU) and the edge (laptop). The laptop

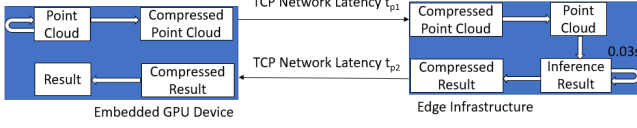


Figure 1: Schematic of our testbed experimental setup.

Model	Lib	Method	MAC	#Param	Acc
RandLA-Net [9]	PT	Point	362B	1.24M	52.6%
RandLA-Net [9]	TF	Point	66.5B	1.24M	53.9%
3D-MiniNet [2]	PT	Proj	N/A	4M	55.8%
3D-MiniNet [2]	TF	Proj	N/A	4M	55.8%
PolarNet [27]	PT	Proj	136B	14M	54.3%
RangeNet++ [18]	PT	Proj	378B	50M	52.2%
RangeNet++ [18]	TRT	Proj	N/A	50M	51.9%

Table 2: Multiply and accumulate operations (MAC), number of Parameters (#Param) and accuracy (Acc) [1] of each model.

is running Ubuntu 18.04 (Intel i7-7500U CPU @ 2.70GHz, 8GB Samsung 2133MHz RAM) via a direct wired Gigabit Ethernet connection. We use Linux Traffic Control (TC) with NetEm to emulate different network conditions (upstream and down stream bandwidth and latency). The performance of model inference on edge is again emulated regarding a Google Cloud Compute Server with a NVIDIA T4 GPU and artificial latency is added to laptop processing. This delay is the same for all models (30ms), and is based on running PolarNet [27].

Depending on the inference and edge offloading strategy considered within each experiment, a point cloud data compression module (using Octree [17]), can be switched on or off. Monitors are placed on embedded devices to measure its power consumption and model inference latency.

3.2 SOTA Semantic Segmentation Models

We consider a representative set of SOTA point cloud segmentation models (Table 2) for exploring the benefit of edge assisted processing of LIDAR point cloud data. These models use different semantic segmentation techniques (point and projection based approaches) and are implemented using different DNN frameworks/libraries – TensorFlow (TF), PyTorch (PT), TensorRT (TRT). Where possible, the same model is evaluated using different frameworks with the intention of making our conclusions as framework independent as possible.

It is however important to note that due to the differences in how models are realized in TF (particularly TF1) and PT, direct comparisons should not be made between the properties of these models. Additionally, multiply and accumulate (MAC) operations represent the number of multiply/accumulate operations within a model, a common task within DNNs which generally employ matrix vector maths.

3.3 Dataset

We use Semantic-KITTI [4], a widely used 3D segmentation benchmarking dataset, for evaluation and summarize it in Table 3. This dataset is aimed at vision training of autonomous vehicles, providing an understanding of objects and surfaces surrounding a moving

Rate	#Classes	Labeling	#Points	#Scans	#Seq
10 Hz	28	Pointwise	4.5B	43,552	21

Table 3: Semantic-Kitti dataset summary.

vehicle, and contains sequences of real-world LIDAR generated point clouds. The realism, data size and well-established nature of Semantic-KITTI makes 3D-Segmentation and energy efficient model inference a challenging task. All tests were run on the official evaluation sequence: *Sequence 8*.

Semantic-KITTI point clouds contain both positional co-ordinate information and a float intensity value. As such, the point clouds do not perfectly suit the standard Octree compression algorithm which is designed for point clouds with either only positional information or positional information alongside an integer RGB color value. A workaround was devised such that the intensity value was encoded into the red component of the RGB variable of Octree compression. There is a slight loss of information due to float inaccuracies, as the float must be stored within an 8-bit integer.

3.4 Inference Strategies

We use on-device model inference as the baseline for assessing the benefits with edge offloading. In particular, we consider two edge offloading strategies: 1) offloading without sensor data compression; 2) offloading with sensor data compression. Comparison between these strategies can reveal the cost of point cloud data communication and the need for some form of data compression.

On-device Processing. The first strategy considers keeping inference of each of the chosen models fully on-device, allowing the latency and energy costs for each model to be determined. Care is taken to ensure that the GPU is fully utilized during the execution of each model for the best performance results. With this strategy, compression and the edge-offloading module (in Figure 1) are disabled.

Edge Offloading. The second strategy emulates a basic edge-assisted 3D semantic segmentation scenario. TC is used to emulate a range of network conditions between “ideal” and “poor” through control of bandwidth (between 25Mbps and 250Mbps) and latency (10ms or 30ms) as per [15]. This reflects the setting whereby inference takes place on the edge as opposed to the embedded device, with raw point clouds streamed to the edge for processing, and results returned.

Edge Offloading with Compression. This third strategy is a variant of the above edge-offloading strategy augmented with point cloud data compression. Specifically, we apply the Octree compression algorithm [19] prior to offloading, and a simple zip compression to the returning inference results (binary labels). Octree algorithm has a number of tunable parameters (such as Octree Resolution and Point Resolution) in order to provide flexibility in compressive ratio, compression speed, and ultimately accuracy. Due to space restrictions, only the best performing Octree configuration in terms of inference latency is shown.

3.5 Evaluation Metrics

We consider two key metrics to evaluate the benefit of edge offloading to point cloud segmentation: 1) *inference latency (IL) in seconds*; 2) *energy per inference (EI) in Joules*. These metrics shown in our

results are an average of multiple experiments corresponding to each data point.

Inference Latency (IL). IL refers to the full end-to-end time to process a point cloud frame, averaged across all frames in the sequence. For on-device processing, Hadidi et al. [7] had a similar methodology. For the two edge offloading cases, network transmission and any processing also contributes to the total IL.

Energy per Inference (EI). For on-device processing, EI is defined as the inference latency multiplied by the average power consumption, as in [7]. For the edge offloading cases, EI is the total power consumption across all inferences divided by frames per second (FPS). Here FPS denotes the average number of point cloud frames that can be processed every second. A differing calculation is required as there can be multiple frames within the edge processing pipeline at any time. Power consumption sampled at 25Hz, and is measured based on the entire power usage on the embedded GPU (including any time waiting for inference results in the case of offloading).

4 RESULTS

4.1 On-device Processing

Through evaluation of each SOTA model on-device, it can be observed that inference performance is approximately proportional to MAC and device capability. Additionally, real-time processing cannot be achieved (§4.1), and greater computational capacity does not always translate to ultimate efficiency (§4.1).

Inference Latency (IL). Figure 2 details the IL against model for each of the devices. All models, regardless of device, have an IL too large to be considered real-time. This motivates an opportunity to offload the models in some way to allow the strict real-time requirements to be met on resource constrained devices. We have additionally evaluated a smaller version of the 3D-MiniNet model (3D-MiniNet-Tiny) on the TX2. This model has only 0.4M parameters, and achieves a non-SOTA accuracy of 49.0%. Even with this tiny model, the best achievable inference was 45ms – approximately 12ms over our real-time inference goal.

Energy per Inference (EI). Figure 3 shows the EI for each model (on each device), and its results provide similar motivation to offload, as well as providing a straw-man baseline when offloading. Note that GPU vs CPU power split could not be measured on the Jetson Nano. The EI of the PyTorch RangeNet++ is significantly higher than any other model. Firstly, compared to other models, its MAC value is very large (for a projection-based approach). This significantly impacts the number of operations which must be completed per inference. Secondly, the model’s inference latency is high, while the GPU and CPU power usage is not (Figure 4); in combination this results in a high EI.

Furthermore, the GPU and CPU usage of RandLA-Net is higher than any other model regardless of framework. RandLA-Net is the only model following a point-based approach, which contains a very expensive KNN step to find point neighbors. The expectation is that Tensor-RT will not only make the PyTorch version of a model more (energy) efficient, but it will also provide a speed boost (with a slight cost to accuracy). For the Range-Net++ model, however, only energy improvements were observed.

BW (Mbps)	Network (ms)	Xavier EI (J)	TX2 EI (J)	Nano EI (J)
250	10	0.27	0.15	0.14
100	10	0.69	0.30	0.27
50	10	2.03	0.91	0.76
25	10	4.29	1.67	1.41
250	30	0.31	0.17	0.16
100	30	1.02	0.43	0.24
50	30	2.73	1.33	0.99
25	30	7.33	3.38	2.51

Table 4: Energy per inference with edge offloading for each device and different network conditions.

4.2 Edge Offloading

We next report the results of a basic form of edge offloading where raw point cloud data is streamed to the edge under varying network conditions.

Inference Latency (IL). We observe that even under perfect network conditions, this way of edge offloading is insufficient to achieve real-time processing. Specifically, the best achievable inference latency in this scenario is only 0.38s on the AGX Xavier, using the PolarNet model (Figure 5). Although worse than on-device processing, it should also be noted that in the case of Jetson TX2, similar latencies are observed, and for the Jetson Nano (the most resource constrained device) improvements can be clearly witnessed.

Energy per Inference (EI). Promisingly, we see significant improvements in EI across the board, with the best conditions on the AGX Xavier showing 91.5% improvements over the best on-device processing, and with the other devices achieving similar improvements of around 90% (Table 4). Unsurprisingly, we also observe that better network conditions usually yield a lower energy consumption as it reduces the time spent communicating the point cloud data to the edge, and additionally the time waiting for inference results is returned.

In summary, when the basic form edge offloading is applied, although the energy per inference is greatly improved, model inference is still far from meeting the real-time requirements, even in ideal network conditions.

4.3 Edge Offloading with Compression

Finally we report the performance of edge offloading coupled with data compression. Depending on the compression parameters used, the size of the point cloud files could be compressed from around 2MB to between 305 and 585 KB, with the resultant loss and compression time also dependent on these parameters (with loss increasing as the Point Resolution value was increased). Table 5 outlines the offloading results utilizing the best performing compression parameters (in terms of inference latency), while ensuring negligible loss in resultant model accuracy following compression. There is no single optimal compression setting which suits all network conditions; rather it is dependent on compression time and resultant file size.

The main takeaways here are that significant improvements can be observed in terms of both IL and EI. However, even with idealistic conditions, this approach is unable to obtain close to real-time processing on any device or network conditions. As expected

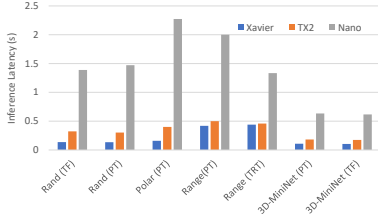


Figure 2: Inference latency with on-device processing for each model.

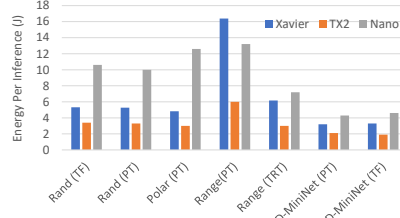


Figure 3: Energy per inference with on-device processing for each model.

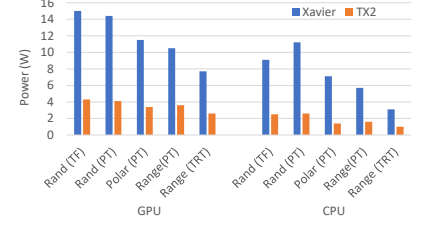


Figure 4: GPU and CPU power consumption with on-device processing for each model

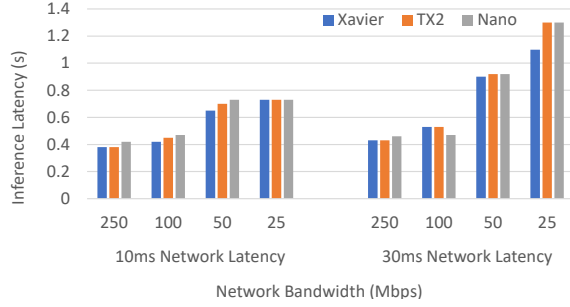


Figure 5: Inference latency with edge offloading for PolarNet model and different devices.

Condition	Device	PR	OR	IL (s)	EI (J)
Ideal	Xavier	0.01	1	0.092	0.07
	TX2		1	0.104	0.05
	Nano		1	0.146	0.047
Poor	Xavier	0.01	0.5	0.345	0.39
	TX2		0.5	0.386	0.26
	Nano		0.5	0.410	0.21

Table 5: Compressive Edge Offloading results from ideal (250Mbps Bandwidth and 10ms Network Latency) and poor (25Mbps Bandwidth and 30ms Network Latency) network conditions. OR represents Octree Resolution. PR represents point resolution.

performance begins to degrade as network conditions do; however it does show strong promise as a technique which could contribute towards real-time point cloud processing (Table 5).

The Jetson Nano is the biggest winner from compressed edge offloading. Even in the worst network conditions, it is still able to achieve an inference latency of 0.410s, a 32.8% and 81.9% improvement, respectively, over the best and worse inference latencies with on-device processing, with negligible loss in accuracy.

The EI is vastly improved compared to on-device inference. For the most efficient model, on-device can achieve 2.1J, 1.6J and 4J on AGX Xavier, Jetson TX2 and Jetson Nano, respectively. With compressive offloading, the worst achieved are 0.39J, 0.26J and 0.21J, respectively. EI, when compared to the uncompressed offloading, is similar and only marginal gains are made, implying that the addition of compression does not add too much overhead to device energy consumption.

5 DISCUSSION AND FUTURE WORK

Our results above have shown that even by offloading with compression and under idealistic conditions, the real-time inference latency goal cannot be met, although massive improvements in terms of energy efficiency are achieved with offloading regardless of condition or device. Future work in this field should *focus on improving the offloading methodology to make progress towards real-time processing, with a particular focus on suiting a wide range of devices and network conditions*. In foresight, there is also an urgent need to find solutions to cope with further expected increases in LIDAR density over the coming years. Cutting-edge LIDAR today is able to produce upto 5x more points than five years ago, reaching densities of 200,000 points per frame; this upward trend is likely to continue. This, in turn, will require larger DNNs to segment this data. The increasing popularity of LIDAR will also result in additional pressure to have mature techniques in place to effectively process LIDAR data in *real-time and at scale*. Here we chart some future directions and other related application domains of large-scale point cloud segmentation.

In addition to these, there have been promising complimentary work completed in both compression [21] and hardware acceleration [12] with could benefit the work completed here. The acceleration can help us to achieve realtime inference on particular model size (e.g., 3D-MiniNet). Its commonly know that the growth of data rate and computation demand from DNNs exceed the acceleration from algorithms and hardware, making it necessary to seek for a distributed solution from system point of view.

5.1 Dynamic Model Partitioning

A promising direction to explore is model partitioning, where the computation (layers) of a DNN are split across device and edge sequentially as a head and a tail. This takes edge offloading a step further, by leveraging the computational power of both the edge and the end device, by only offloading when deemed necessary. Neurosurgeon [10] dynamically determines a model partitioning point based on network conditions but only considers models in the computer vision (images), speech, and NLP domains. It would be interesting to explore how dynamic model partitioning techniques can be applied to point cloud segmentation tasks in which the DNNs evaluated may have vastly differing operators (e.g., PointConv [23]). The dynamic strategies will also need to take both device capability and model architectures into account – a more powerful device will be able to process more of a model than a less powerful one and as a result can reduce network latency. Additionally, certain

models will be intrinsically suited to this approach if they contain bottleneck layers, where the model naturally narrows resulting in low data volume transmission at the partitioning point. To this end, layer-wise profiling will be needed to determine optimal model partitioning points. Moreover, tradeoffs between offloading sizes and end accuracy need to be taken into account.

5.2 Bespoke DNN Architectures from Construction Phase

Effective model partition strategies seldom come alone but often require suitable DNN architectures in conjunction. An interesting direction is to identify which layers in DNNs can serve as an efficient partitioning point. **Bottleneck injection** could be adopted to reduce the intermediate result size in strategic place(s) [16]. This would allow for an optimal splitting point where offloading can occur that minimizes network latency and, if getting to this bottleneck is not computationally demanding, the overall latency may be reduced. However, the requirement of retraining the models once the bottleneck is added reduces the flexibility of such approach, requiring that the artificial bottleneck be determined a priori. Furthermore, model compression through bottleneck injection usually comes with accuracy loss. As far as can be seen, no point cloud based tasks have attempted this technique, and it may be beneficial to explore the benefits that this could provide. However, due to the retraining requirements, this technique could be impractical on large datasets. The ideas established may be useful in further exploring autoencoders as described below.

Given the size of LIDAR point cloud data, the computational demand for processing the first few layers in a 3D point cloud compatible DNN model may still be an issue for on-device processing. An **imbalanced autoencoder** is a more general framework which enables the encoding of offloaded data into small sizes, with limited computational impact on the end device [25]. The goal of the autoencoder is not to facilitate the complete reconstruction of the data, but to work alongside any existing models to achieve the best possible inference. No alterations need to be made to the original model making it easily applicable to a range of different models. It is worth establishing whether this approach to offloading is preferable to compressive offloading, and in what conditions. Additionally, it may be worth corroborating the claims made in [25] regarding the appropriate offloading point in splitting the processing across the end device and the edge, when considering LIDAR point cloud data. Furthermore, exploring whether autoencoders can be placed anywhere within a model may be worthwhile. This would mitigate the need to retrain the entire model for each partition point, as with artificial bottlenecks. If successful, this may have the ability to combine the flexibility of autoencoders/model partitioning with significant reduction in data sizes at partition points.

5.3 More Complicated Segmentation Tasks

In addition to semantic segmentation, **instance segmentation** and **panoptic segmentation** on point clouds are alternative applications worth investigating in future work. These tasks are known for their greater complexity over semantic segmentation and it would be interesting to determine if these point cloud data applications

differ significantly in their latency and energy consumption characteristics and the gains various edge offloading techniques can bring.

REFERENCES

- [1] [n. d.]. Papers with code - semantickitti benchmark (3D semantic segmentation). <https://paperswithcode.com/sota/3d-semantic-segmentation-on-semantickitti>
- [2] Inigo Alonso, Luis Riazuelo, Luis Montesano, and Ana C. Murillo. 2020. 3D-MiniNet: Learning a 2D Representation From Point Clouds for Fast and Efficient 3D LIDAR Semantic Segmentation. *IEEE Robotics Autom. Lett.* (2020).
- [3] Amin Banitalebi-Dehkordi, Naveen Vedula, Jian Pei, Fei Xia, Lanjun Wang, and Yong Zhang. 2021. Auto-Split: A General Framework of Collaborative Edge-Cloud AI. In *KDD*.
- [4] Jens Behley, Martin Garbade, Andres Milioto, Jan Quenzel, Sven Behnke, Cyrill Stachniss, and Jürgen Gall. 2019. Semantickitti: A Dataset for Semantic Scene Understanding of LIDAR Sequences. In *ICCV*.
- [5] Jiasi Chen and Xukan Ran. 2019. Deep Learning With Edge Computing: A Review. *Proc. IEEE* (2019).
- [6] Yulan Guo, Hanyun Wang, Qingyong Hu, Hao Liu, Li Liu, and Mohammed Bennamoun. 2021. Deep Learning for 3D Point Clouds: A Survey. *IEEE Trans. Pattern Anal. Mach. Intell.* (2021).
- [7] Ramyad Hadidi, Jia Shen Cao, Yilun Xie, Bahar Asgari, Tushar Krishna, and Hye-soon Kim. 2019. Characterizing the Deployment of Deep Neural Networks on Commercial Edge Devices. In *IISWC*.
- [8] Qingyong Hu, Bo Yang, Sheikh Khalid, Wen Xiao, Niki Trigoni, and Andrew Markham. 2021. Towards Semantic Segmentation of Urban-Scale 3D Point Clouds: A Dataset, Benchmarks and Challenges. In *IEEE CVPR*.
- [9] Qingyong Hu, Bo Yang, Linhai Xie, Stefano Rosa, Yulan Guo, Zhihua Wang, Niki Trigoni, and Andrew Markham. 2020. RandLA-Net: Efficient Semantic Segmentation of Large-Scale Point Clouds. In *IEEE/CVF CVPR*.
- [10] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor N. Mudge, Jason Mars, and Lingjia Tang. 2017. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. In *ASPLOS*.
- [11] Li Lin, Xiaofei Liao, Hai Jin, and Peng Li. 2019. Computation Offloading Toward Edge Computing. *Proc. IEEE* (2019).
- [12] Yujun Lin, Zhekai Zhang, Haotian Tang, Hanrui Wang, and Song Han. 2021. PointAcc: Efficient Point Cloud Accelerator. In *MICRO '21*.
- [13] Luyang Liu, Hongyu Li, and Marco Gruteser. 2019. Edge Assisted Real-time Object Detection for Mobile Augmented Reality. In *ACM MobiCom*.
- [14] Zhijian Liu, Haotian Tang, Yujun Lin, and Song Han. 2019. Point-Voxel CNN for Efficient 3D Deep Learning. In *NeurIPS*.
- [15] Ayesha Abdul Majeed, Peter Kilpatrick, Ivor T. A. Spence, and Blessen Varghese. 2020. Modelling Fog Offloading Performance. In *ICFEC*. IEEE.
- [16] Yoshitomo Matsubara, Marco Levorato, and Francesco Restuccia. 2021. Split Computing and Early Exiting for Deep Learning Applications: Survey and Research Challenges. *CoRR* (2021).
- [17] Donald Meagher. 1982. Geometric Modeling Using Octree Encoding. *Comput. Graph. Image Process.* (1982).
- [18] Andres Milioto, Ignacio Vizzo, Jens Behley, and Cyrill Stachniss. 2019. RangeNet ++: Fast and Accurate LIDAR Semantic Segmentation. In *IROS*.
- [19] Ruwen Schnabel and Reinhard Klein. 2006. Octree-based Point-Cloud Compression. In *PBG@SIGGRAPH*.
- [20] Shaoshuai Shi, Chaoxu Guo, Li Jiang, Zhe Wang, Jianping Shi, Xiaogang Wang, and Hongsheng Li. 2020. PV-RCNN: Point-Voxel Feature Set Abstraction for 3D Object Detection. In *CVPR*.
- [21] Dorina Thanou, Philip A. Chou, and Pascal Frossard. 2016. Graph-Based Compression of Dynamic 3D Point Cloud Sequences. (2016).
- [22] Vivek Venugopal and Suresh Kannan. 2013. Accelerating real-time LIDAR data processing using GPUs. In *MWSCAS*. IEEE.
- [23] Wenxuan Wu, Zhongang Qi, and Fuxin Li. 2019. PointConv: Deep Convolutional Networks on 3D Point Clouds. In *CVPR*. Computer Vision Foundation / IEEE.
- [24] Chenfeng Xu, Bichen Wu, Zining Wang, Wei Zhan, Peter Vajda, Kurt Keutzer, and Masayoshi Tomizuka. 2020. SqueezeSegV3: Spatially-Adaptive Convolution for Efficient Point-Cloud Segmentation. In *ECCV*.
- [25] Shuocho Yao, Jinyang Li, Dongxin Liu, Tianshi Wang, Shengzhong Liu, Huajie Shao, and Tarek F. Abdelzaher. 2020. Deep Compressive Offloading: Speeding Up Neural Network Inference by Trading Edge Computation for Network Latency. In *SenSys*.
- [26] Xumiao Zhang, Anlan Zhang, Jiachen Sun, Xiao Zhu, Y. Ethan Guo, Feng Qian, and Z. Morley Mao. 2021. EMP: Edge-Assisted Multi-Vehicle Perception. In *ACM MobiCom*.
- [27] Yang Zhang, Zixiang Zhou, Philip David, Xiangyu Yue, Zerong Xi, Boqing Gong, and Hassan Foroosh. 2020. PolarNet: An Improved Grid Representation for Online LIDAR Point Clouds Semantic Segmentation. In *CVPR*.
- [28] Li Zhou, Hao Wen, Radu Teodorescu, and David H. C. Du. 2019. Distributing Deep Neural Networks with Containerized Partitions at the Edge. In *HotEdge*.